After clicking the *Upload* button, the server will check the extension, think an image is being uploaded, and return a message like 'Thank you for uploading your image!' Once the file is uploaded, the malicious code can be executed on the server.

Another popular way of securing file upload forms is to protect the folder where the files are uploaded using a *.htaccess* file to restrict execution of script files in this folder. A typical '*.htaccess*' file for this will contain the following code:

```
AddHandler cgi-script .php .php3 .php4 .phtml .pl .py .jsp
.asp .htm .shtml .sh .cgi
Options -ExecCGI
```

The above snippet is a type of blacklist approach, which in itself is not very secure. An attacker can easily bypass such checks by uploading a file called ".htaccess ", which contains a line of code such as:

```
AddType application/x-httpd-php .jpg
```

This line instructs Apache to execute *.jpg* images as if they were PHP scripts. The attacker can now upload a file with a *.jpg* extension, which contains PHP code. Because uploaded files can and will overwrite existing ones, malicious users can easily replace the *.htaccess* file with their own modified version, allowing them to execute specific scripts that can help them compromise the server.

Time for security

File upload is an essential functionality in most Web applications, and with the following measures, they can be designed securely:

1. Do not use the user-supplied file name as a file name on your local system. Instead, create your own unpredictable file name. Something like a hash (md5/sha) works, as it is easily validated (it is just a hex number). Maybe add a serial number or a time-stamp to avoid accidental collisions.
2. Particularly if uploaded files are viewable by others without moderator review, they have to be authenticated. This way, it is at least possible to track who uploaded the objectionable file.
3. Log the details (such as time, client's IP address and user name) of file uploads and other related events. They can help you check what types of attacks have been made against your server, and whether any were successful.
4. If possible, also try a malware scan of uploaded files; upload the files in a directory outside the server root.
5. Don't rely on client-side validation only for uploading files, since it is not enough. Ideally, one should have both server-side and client-side validation implemented.

Note: I once again stress that neither LFY nor I are responsible for the misuse of the information given here. Rather, the attack techniques are meant to give you the knowledge that you need to protect your own infrastructure. Please use the tools and techniques given above, sensibly.

We will deal with other dangerous attacks on Web applications and Apache in the next article. Always remember: Know hacking, but no hacking. END

Further reading

[1] *http://www.martinsecurity.net/2009/05/14/remote-file-inclusion-attacks-pt1remote-file-inclusion-attacks-pt1/*

By: Arpit Bajpai

The author is a FOSS enthusiast, and loves to troubleshoot in the information security domain, while exploring new exploits and system vulnerabilities. You can send your queries and constructive feedback on the article to *abajpai75 AT yahoo DOT com*.